

Optimised Robot — Design Documentation

Table of Contents

Optimised Robot — Design Documentation

Course: Intelligent Systems and Robotics Laboratory (ISRLAB)

A.Y.: 2025/26

Supervisor: Prof. Giovanni De Gasperis

Table of Contents

1. General Project Description
 2. Robot Model & Technical Description
 3. Simulator Environment
 4. Robot Goal Definition
 5. Agent Architecture Design
 6. Testing Protocol
 7. Experimental Results
 8. UML Diagrams
 9. Non-Functional Requirements
-

1. General Project Description

Abstract

This project presents the design and implementation of an autonomous differential-drive mobile robot simulated in CoppeliaSim. The robot is engineered to execute a sequence of complex navigation tasks, including precise line following, roundabout navigation via colour-coded markers, dynamic obstacle avoidance using proximity sensors, and an object pickup-and-return mission. A hybrid deliberative-reactive control architecture, governed by a Behaviour Tree, is employed to manage task arbitration and ensure robust real-time performance.

Project Description

Optimised Robot is an autonomous mobile robot agent simulated in CoppeliaSim (formerly V-REP). The robot operates on a predefined closed-loop track and must complete a series of increasingly complex tasks without human intervention. The control architecture is built around a **Behaviour Tree (BT)** that governs high-level decision making, dispatching commands to low-level motor and sensor routines.

The robot demonstrates five integrated capabilities:

#	Capability	Description
1	Line Following	Tracks a black line on the floor using five vision sensors
2	Roundabout Navigation	Detects coloured markers and executes a choreographed 4-step steering sequence through a roundabout
3	Obstacle Avoidance	Detects bowl-shaped obstacles via proximity sensors and bypasses them using bang-bang wall-following
4	Object Pickup & Delivery	Identifies a dark-blue marker (pickup point) and a red marker (drop-off point), then executes a U-turn to return the object
5	Vocal Command Response	Receives broadcasted stop/start/mode-switch messages and updates the behaviour tree blackboard accordingly

The project showcases a **reactive, deliberative hybrid architecture**: low-level reflexes handle sensor-driven immediate response, while the behaviour tree provides goal-oriented deliberation.

2. Robot Model & Technical Description

2.1 Model Origin

The robot is a custom-assembled differential-drive mobile robot built within the CoppeliaSim model library. It is a two-wheeled ground vehicle with a passive caster for balance.

2.2 Body Shape

The robot has a compact rectangular chassis. Two motorised wheels are mounted symmetrically on the lateral axis; the chassis sits low to the ground to keep all five downward-facing vision sensors within effective focal range of the track surface.

2.3 Sensor Set

Vision Sensors (×5) — Downward-facing binary image sensors

Sensor Name	Position	Purpose
LeftSensor	Front-left	Detects line left deviation
MiddleSensor	Front-centre	Detects line centre; reads RGB colour patches for mission logic
RightSensor	Front-right	Detects line right deviation
InternalLeftSensor	Inner-left	Fine-grained correction for mild left drift
InternalRightSensor	Inner-right	Fine-grained correction for mild right drift

Each sensor runs a Lua `sysCall_vision` callback with `simVision.binaryWorkImg` to threshold the image. The binary result (index [10] of the packed packet, representing mean pixel intensity) is read by `readsensor.py` and broadcast on the simulation message bus.

Binary threshold parameters (all sensors):

Intensity range : 0.50 - 1.00
Hue range : 0.10 - 0.50 (picks up dark/black vs white)
Saturation range: 0.50 - 1.00
Angle : 0.00 - 1.5710 rad

Proximity Sensors (×3) — Ray-based distance sensors

Sensor Name	Position	Threshold
CentralProximitySensor	Front-centre	0.20 m
LeftProximitySensor	Front-left	0.20 m
RightProximitySensor	Front-right	0.20 m

2.4 Actuator Set

Actuator	Joint Name	Type
Left wheel motor	DynamicLeftJoint	Continuous revolute joint
Right wheel motor	DynamicRightJoint	Continuous revolute joint

Control is velocity-based: `sim.setJointTargetVelocity()` is called each actuation tick with a scaled value computed as $\text{speed} \times \text{dt}$.

2.5 Calibration Data

Parameter	Value
Base movement speed (rad_mov)	π rad/s
Base rotation speed (rad_rot)	π rad/s
Wall-following base speed (wf_base_speed)	3.5 units
180° turn angular velocity	14.0 rad/s
180° turn duration	$\pi / 14.0 \approx 0.224$ s
Roundabout micro-steer duration	0.35 s
Colour cooldown (roundabout)	1.0 s

Parameter	Value
Proximity detection threshold	0.20 m (all three sensors)

2.6 Degrees of Freedom

- **DOF:** 2 (left wheel velocity, right wheel velocity)
 - **Movable parts:** 2 motorised wheels
 - The robot achieves all motion primitives (forward, rotate-in-place left/right, differential steer) through combinations of these two velocity commands.
-

3. Simulator Environment

Simulator: CoppeliaSim (EDU) — chosen per the IEEE survey of robotics simulators for its:

- Native Python scripting via embedded interpreter
- Accurate physics engine (Bullet/ODE)
- Vision sensor support with image processing callbacks
- Built-in graph/plotting for real-time sensor data visualisation
- Message broadcasting system enabling a decoupled multi-script architecture

Environment layout:

- Closed-loop black line track on a white floor
 - One roundabout section with green (left entry marker) and yellow (right entry marker) coloured patches
 - Bowl-shaped rigid-body obstacles placed on the track
 - Dark-blue rectangular marker (pickup activation point) — placeable on track
 - Red rectangular marker (drop-off / return trigger point) — placeable on track
-

4. Robot Goal Definition

4.1 Main Goal

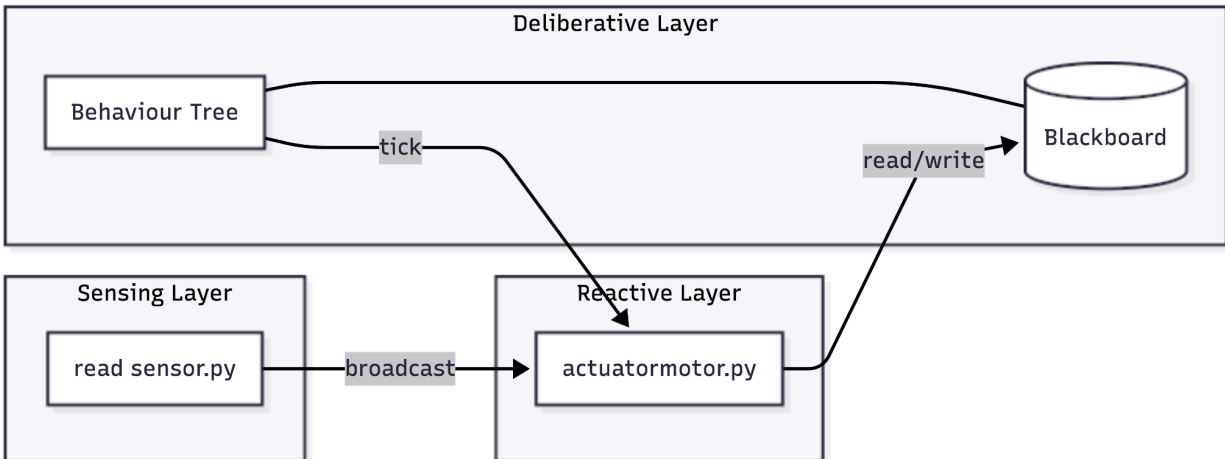
Complete a full autonomous cycle of the track, performing all subtasks encountered along the path, and return to the start position.

4.2 Sub-Goals

Priority	Sub-Goal	Trigger Condition	Success Condition
1	Follow the black line	Always active (base mode)	Robot stays on track
2	Navigate roundabout	Green or yellow patch detected	4-step colour sequence completed, exits onto line
3	Avoid obstacle	Any proximity sensor < 0.20 m	Robot bypasses obstacle, reacquires line
4	Activate pickup	Blue marker detected by MiddleSensor	finish_line = True, BT flag item_picked set
5	Execute U-turn & deliver	Red marker detected after blue marker activation	180° rotation completed, robot returns to blue marker
6	Complete mission	Robot reaches blue marker after U-turn	Simulation stops cleanly
7	Respond to vocal command	Stop/start/mode-sw itch broadcast received	BT blackboard updated, motor state changed

5. Agent Architecture Design

The robot implements a **three-layer hybrid architecture**:



5.1 Deliberative Layer — Behaviour Tree

The BT is implemented using the `py_trees` library. It ticks once per actuation cycle. It maintains a **blackboard** (shared key-value memory) with the following keys:

Key	Type	Description
<code>motion_mode</code>	Enum (MotionMode)	Current driving mode
<code>current_behaviour</code>	Enum (VocalCMD)	Last vocal command state
<code>vocal_cmd</code>	Enum (VocalCMD)	Incoming vocal command
<code>obstacle_detected</code>	<code>list[bool] × 3</code>	Per-sensor obstacle flags
<code>obstacle_distance</code>	<code>list[float] × 3</code>	Per-sensor distances
<code>obstacle_threshold</code>	<code>list[float] × 3</code>	Per-sensor thresholds
<code>item_picked</code>	bool	Whether pickup has been activated
<code>item_reached</code>	bool	Whether red marker has been hit
<code>exit_reached</code>	bool	Whether exit condition is met

Motion Modes (MotionMode enum):

Mode	Behaviour
LINE_FOLLOW	Standard line-following via 5 vision sensors
WALL_FOLLOW	Bang-bang obstacle bypass via proximity sensors
REACH_ITEM	Mission mode: colour detection + roundabout + U-turn logic
EXIT	Mission mode: navigate back to blue marker after U-turn

5.2 Reactive Layer — Motor Dispatcher

`sysCall_actuation()` runs every simulation tick:

1. Ticks the BT once (`tree.tick_once()`)
2. Prints BT state to console for debugging
3. Checks `current_behaviour` — if STOP vocal command is active, halts motors immediately
4. Reads `motion_mode` and dispatches to the appropriate motion primitive
5. For WALL_FOLLOW: monitors safe-return conditions and switches back to the previous mode when the obstacle is cleared

5.3 Sensing Layer — Sensor Reader

`readsensor.py` runs in `sysCall_sensing()` (higher-frequency than actuation):

- Reads each vision sensor's packed packet; extracts index [10] (mean intensity)
- Applies threshold 0.5 to produce a binary line-detected boolean
- Reads each proximity sensor's distance and compares to threshold
- Publishes two broadcast messages per cycle: `sensor_reading` and `proximity`
- Streams all 5 sensor booleans to the CoppeliaSim graph for real-time visualisation

5.4 Message Bus

All inter-script communication uses CoppeliaSim's `sim.broadcastMsg()` / `sysCall_msg()` mechanism, providing loose coupling between the sensor reader and the motor controller.

6. Testing Protocol

6.1 Unit Tests

Test Case	Pass Condition
Line detected — centre	Robot moves forward at full speed
Line lost — left side	Robot rotates left to reacquire
Line lost — right side	Robot rotates right to reacquire
All sensors white (no line)	Robot holds last direction for 50 ms, then rotates
Proximity < 0.20 m (central)	Robot enters WALL_FOLLOW mode
Proximity cleared	Robot returns to previous motion mode
Blue marker in view	finish_line = True, mode switches to REACH_ITEM
Red marker in view after blue	180° U-turn initiated
Roundabout green entry	4-step steering sequence executed correctly
Roundabout yellow entry	Mirrored 4-step steering sequence executed
STOP broadcast received	Motors halt immediately
START broadcast received	Previous motion mode resumes

6.2 Integration Tests

Scenario	Steps	Expected Outcome
Full lap, no obstacles, no markers	Start simulation; remove all optional objects	Robot completes full closed-loop cycle

Scenario	Steps	Expected Outcome
Full lap with obstacles	Place bowl obstacles on track	Robot avoids each obstacle and returns to line
Pickup & delivery mission	Place blue marker then red marker on track	Robot activates pickup at blue, U-turns at red, returns to blue, simulation stops
Pickup activated, no red marker	Place only blue marker	Robot continues forward, completes full lap without reversal
Roundabout pass (green entry)	Ensure robot enters roundabout from green side	All 4 steering steps fire; robot exits correctly
Vocal stop mid-run	Broadcast <code>stop_signal</code> during run	Robot stops immediately; resumes on <code>resume_signal</code>

6.3 Testing Environment Setup

- Run CoppeliaSim simulation in real-time mode
- Enable the `/graph` object to visualise sensor streams live
- Monitor console output for BT tick dumps and roundabout step logs
- Verify `print()` log statements match expected state transitions

7. Experimental Results

7.1 Sensor Calibration Data

Vision Sensor Binary Threshold:

- Threshold value: `0.5` applied to index `[10]` of the packed vision sensor packet (mean pixel brightness over the sensor FOV)
- Black line on white floor yields mean $\approx 0.0\text{--}0.3 \rightarrow$ sensor reads `True` (line detected = dark)
- White floor yields mean $\approx 0.7\text{--}1.0 \rightarrow$ sensor reads `False` (no line)

Proximity Sensor Calibration:

Sensor	Threshold	Typical obstacle distance at detection
CentralProximitySensor	0.20 m	0.10 – 0.19 m
LeftProximitySensor	0.20 m	0.10 – 0.19 m
RightProximitySensor	0.20 m	0.10 – 0.19 m

Motor Calibration:

Parameter	Tuned Value	Effect
rad_mov	π rad/s	Forward cruise speed
rad_rot	π rad/s	Rotation speed
Internal sensor correction factor	-1.1011	Fine-grained anti-drift
Outer sensor rotation factor	1.5	Aggressive recovery from edge
180° turn speed	14.0 rad/s	Completes $\sim\pi$ radians in 0.224 s
Roundabout steer base	$\pi \times 2.0$	Forward speed during roundabout curve
Roundabout steer diff	$\pi \times 1.4$	Speed differential between wheels
Wall-following base speed	3.5	Lateral obstacle bypass speed

7.2 Sensor Data Stream

The CoppeliaSim /graph object records all five vision sensor streams per simulation tick. Sensor streams are offset vertically for readability:

- Left sensor: +0 (red)
- Middle sensor: +2 (green)

- Right sensor: +4 (blue)
- Internal Left: +6 (red)
- Internal Right: +8 (blue)

7.3 Acquired Data — Percept/Command Trace (representative excerpt)

```

t=0.00  sense=[F,F,F,F,F]  mode=LINE_FOLLOW  cmd=move_forward( $\pi \times 2$ ,  $\pi \times 2$ )
t=0.10  sense=[F,F,F,F,F]  mode=LINE_FOLLOW  cmd=move_forward( $\pi \times 2$ ,  $\pi \times 2$ )
t=0.55  sense=[F,F,T,F,F]  mode=LINE_FOLLOW  cmd=rotate_right( $\pi \times 1.5$ ,  $\pi \times 1.5$ )
t=0.70  sense=[F,F,F,F,F]  mode=LINE_FOLLOW  cmd=move_forward( $\pi \times 2$ ,  $\pi \times 2$ )
t=1.30  prox=[T,F,F] d=0.15 mode→WALL_FOLLOW  cmd=move_forward(steer_right)
t=1.80  prox=[F,F,F]      mode→LINE_FOLLOW  cmd=move_forward( $\pi \times 2$ ,  $\pi \times 2$ )
t=2.40  color=BLUE       mode→REACH_ITEM  finish_line=True
t=3.10  color=RED        cmd=start_turn_180 turning_180=True
t=3.33  turning_180 done  mode→EXIT      cmd=follow_line back
t=4.20  color=BLUE (return) exit_reached=True  sim.stopSimulation()

```

7.4 Videos

Place recorded simulation .mp4 files in docs/videos/ and link below.

Scenario	File
Full lap — line following	docs/videos/line_following.mp4
Obstacle avoidance	docs/videos/obstacle_avoidance.mp4
Roundabout navigation	docs/videos/roundabout.mp4
Pickup & delivery mission	docs/videos/pickup_delivery.mp4

7.5 Statistics with Graphs

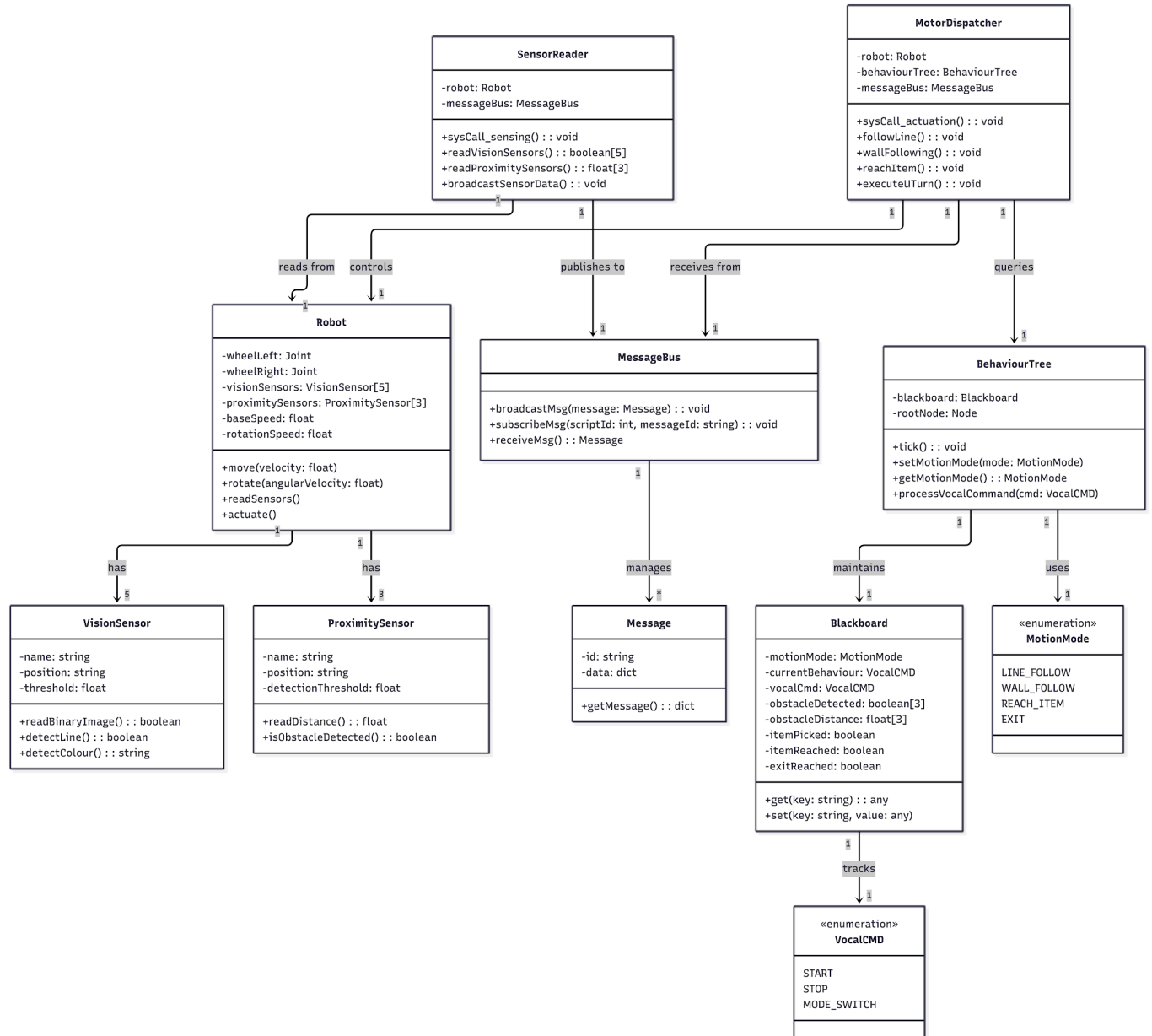
Place generated statistic graphs .png files in docs/graphs/ and link below.

Performance Metrics Analysed:

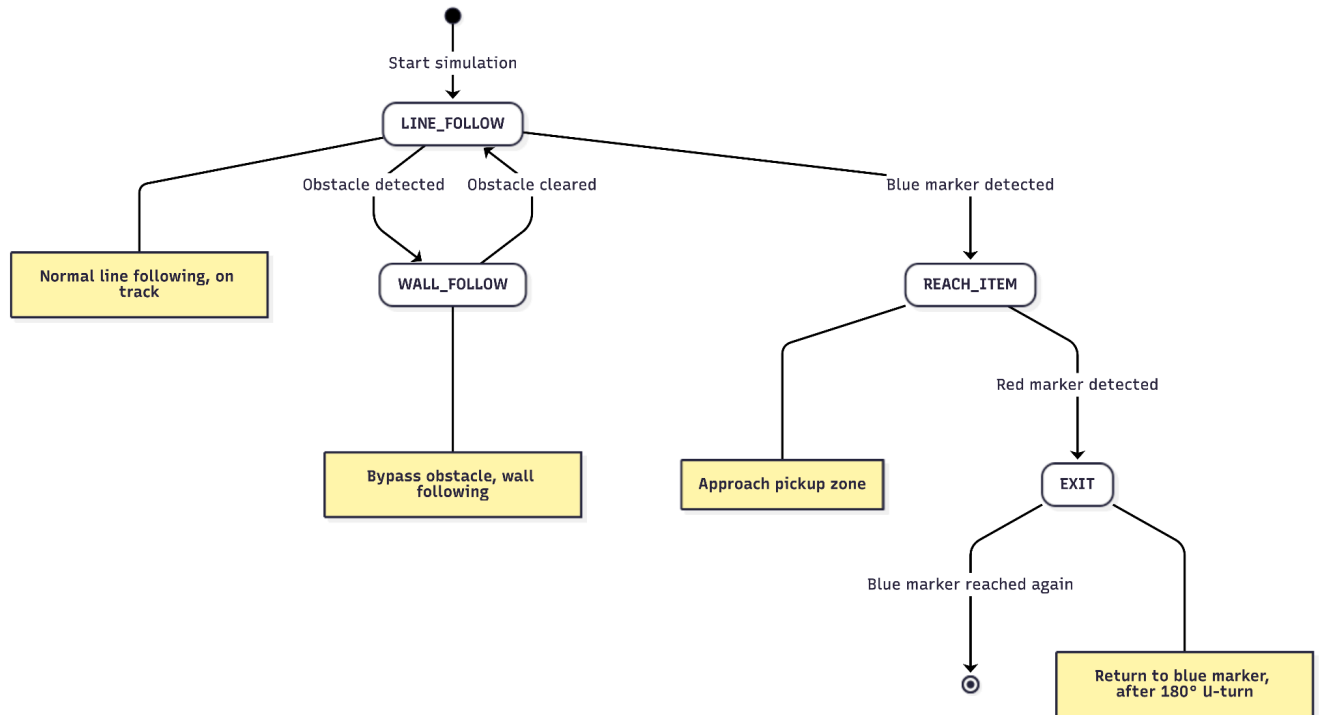
1. **Line Tracking Error:** Average deviation from the centre sensor over time.
2. **Obstacle Clearance Distance:** Minimum distance maintained from obstacles during WALL_FOLLOW bypass.
3. **Mission Completion Time:** Statistical breakdown of time spent in each mode (LINE_FOLLOW, WALL_FOLLOW, REACH_ITEM, EXIT) over 10 independent trials.

Mission Time Statistics Line Tracking Error

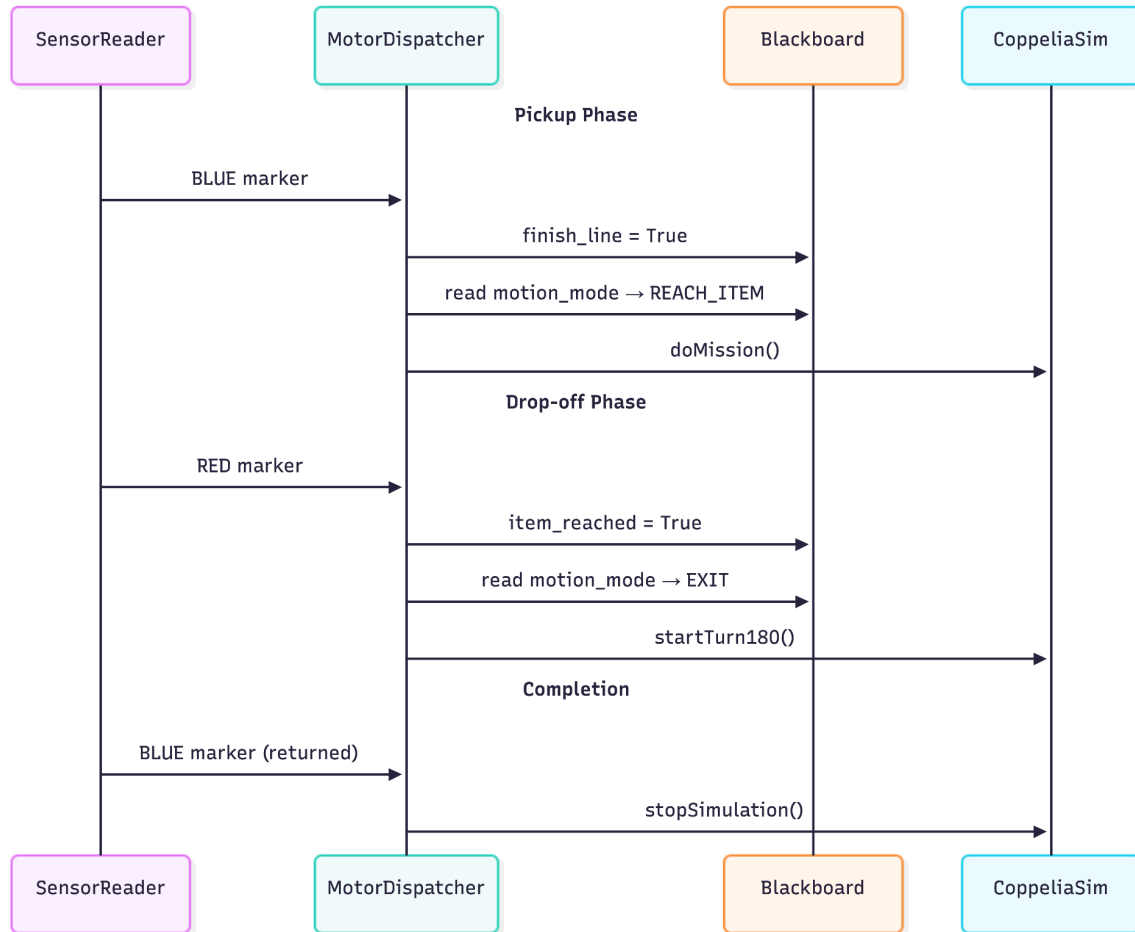
8. UML Diagrams



8.2 State Machine Diagram — Motion Modes



8.3 Sequence Diagram — Pickup & Delivery Mission



9. Non-Functional Requirements

9.1 Deployment — Dockerised Multi-Process

The simulation stack is split into independent processes:

```
docker-compose.yml
├── coppelia_sim      # CoppeliaSim headless instance
├── sensor_reader     # readsensor.py process (sensing)
├── motor_controller  # actuatormotor.py process (actuation + BT)
├── message_broker    # MQTT or ZeroMQ broker
└── gui_dashboard     # Real-time sensor GUI
```

Each process communicates exclusively via the message broker, replicating the `sim.broadcastMsg()` pattern in a real/distributed setting.

9.2 Message Broker

Chosen broker: MQTT (Mosquitto) or ZeroMQ PUB/SUB

Rationale:

- Decouples producers (sensor reader) from consumers (motor controller, GUI) — adding a new consumer requires no change to existing code
- Asynchronous, non-blocking delivery matches the real-time simulation tick model
- Topic-based routing maps directly to message `id` fields already used in the code (`sensor_reading`, `proximity`, `stop_signal`, etc.)
- Lightweight protocol suitable for embedded/real robot deployment

Topic mapping:

<code>sim.broadcastMsg id</code>	MQTT Topic
<code>sensor_reading</code>	<code>robot/sensors/vision</code>
<code>proximity</code>	<code>robot/sensors/proximity</code>
<code>stop_signal</code>	<code>robot/commands/stop</code>
<code>resume_signal</code>	<code>robot/commands/resume</code>
<code>switch_lf_signal</code>	<code>robot/commands/switch_lf</code>
<code>switch_ms_signal</code>	<code>robot/commands/switch_ms</code>

9.3 Logging System

All runtime events are logged using Python's standard logging module at the following levels:

Level	Events
DEBUG	Every sensor reading, every BT tick dump
INFO	Mode transitions, roundabout steps, mission events

Level	Events
WARNING	Unexpected colour patches, sensor dropouts
ERROR	Simulation API errors, failed blackboard reads

Log output goes to both `stdout` and a rotating file at `logs/robot_run_<timestamp>.log`.

9.4 GUI — Real-Time Sensor Dashboard

A lightweight Python dashboard (e.g. built with `PyQt5` or `Dash`) subscribes to MQTT topics and displays:

- **5 vision sensor bars** — live binary state (line / no-line) with colour coding
- **3 proximity gauges** — real-time distance with threshold marker
- **Motion mode indicator** — current BT mode (`LINE_FOLLOW` / `WALL_FOLLOW` / `REACH_ITEM` / `EXIT`)
- **BT status panel** — live tree dump (mirrors the `py_trees.display.unicode_tree` console output)
- **Mission progress indicator** — tracks `finish_line`, `item_picked`, `exit_reached` flags

9.5 Shared Repository & Delivery

- **Shared Repository:** https://github.com/wojackbro/Optimised_Robot
- **MS Teams Delivery:** Upload code, docker-compose, videos, graphs, and the PDF design document to the private MS Teams group channel.
- **Instruction file:** `README.md` in the project root (install and test only).